

Steam Dataset Preparation for a Text-Based Retrieval System

Gonalo Pinto

Faculdade de Engenharia da Universidade do Porto
Porto, Portugal
up202204943@edu.fe.up.pt

Jos  Granja

Faculdade de Engenharia da Universidade do Porto
Porto, Portugal
up202205143@edu.fe.up.pt

Manuel Mo

Faculdade de Engenharia da Universidade do Porto
Porto, Portugal
up202205000@edu.fe.up.pt

Leonardo Ribeiro

Faculdade de Engenharia da Universidade do Porto
Porto, Portugal
up202205144@edu.fe.up.pt

Abstract

This document presents the development of a text-based information retrieval system for the Steam video game platform. The work focused on selecting, preparing, and characterizing a dataset extracted from Steam, which contains both structured and unstructured data suitable for information retrieval. Key tasks included data cleaning, removal of irrelevant attributes, exploratory data analysis, and the definition of a conceptual model. The system design and implementation involved the transformation of the dataset into search-optimized fields, definition of a custom Solr schema, and indexing of a subset of the data. Advanced query techniques, such as field boosting and function-based scoring, were implemented to enhance retrieval relevance. The system’s performance was evaluated against a set of predefined user search scenarios, demonstrating its capability to efficiently and effectively retrieve relevant games based on diverse user needs.

1 Introduction

In an era where large volumes of textual information are generated daily, the ability to efficiently process, structure, and retrieve relevant data is increasingly important.

The video game industry, and specifically the Steam platform, provides a rich domain for information retrieval research due to the availability of extensive metadata, descriptions, and user-generated content.

This work focuses on the preparation and processing of a public dataset from the Steam platform [3], which contains both structured data (e.g., release dates, prices, tags) and unstructured information (e.g., game descriptions), making it highly suitable for information retrieval research.

The work methodology included data cleaning, normalization, exploratory data analysis, and the development of a conceptual model to ensure the dataset consisted of a well-characterized and ready for indexing collection of documents.

Building upon the prepared dataset, a retrieval engine was implemented by indexing the data in Apache Solr [5].

Solr’s flexible schema and query capabilities support complex searches and relevance ranking, which are required to address realistic user scenarios and evaluate retrieval effectiveness. This report describes the pipeline from dataset selection to the implementation and assessment of the retrieval system.

2 Selection of Dataset

When searching for a suitable dataset, the priorities were as follows.

- The necessity for the dataset to originate from a reliable and trustworthy platform
- The relevance and interest of the dataset information to each researcher
- The complexity of the dataset, with greater complexity considered more valuable
- The dataset’s volume, as larger datasets provide more information for analysis

Based on this approach, it was determined that open data exploration was the most effective method for locating suitable datasets, going through many websites like Kaggle, Stanford Large Network Dataset Collection, Google Dataset Search, and Hugging Face Datasets. After some research, the chosen dataset was the [steam-games-dataset](#), from Hugging Face.

The dataset featured the entire library of games from the famous [STEAM](#) platform, its documents are games, and its characteristics can be found in this [Table 1](#). It was created using publicly available code under the [MIT license](#), provided in CSV format, and collected via the Steam API and [Steam Spy](#).

It checked all the priorities: it belonged to a trustworthy platform, each researcher was familiar with the dataset and found it interesting (since each has a big liking for video games), the dataset was composed of different types of data (both structured and unstructured textual data), and included more than 100,000 rows and 40 columns, ensuring sufficient volume and complexity for analysis ([Table 1](#)).

2.1 Dataset Source

The dataset was found in the dataset library of Hugging Face, an American company and open source community that provides a platform to discover, share, and collaborate on machine learning models, datasets and tools, primarily for natural language processing and other domains like computer vision. Hugging Face serves as a central hub for the Machine Learning community. It allows members of the community to host and share ML assets and users to be able of downloading a huge collection of pre-trained models for varying tasks. It also has available tools and APIs to build and deploy AI applications. The chosen dataset was published by FronkonGames [2], the real name being Martin Bustos, a game developer with over 25 years of experience, more than 15 published games, and over 70 Unity assets. Based on this extensive background, the dataset was considered sufficiently credible for use in this project.

More information about Hugging Face can be found in IBM [3] and Coursera [1].

2.2 Dataset Quality

Assessing the data quality of the dataset was necessary to ensure the absence of errors during data analysis and retrieval.

After comparing the dataset with the information available in the STEAM library, it was concluded that the dataset accurately reflects reality.

The dataset was last updated on April 26, 2025; therefore, at the time of writing this report, less than six months have passed since the last update, indicating that the dataset is nearly up to date with the current STEAM games library.

Additionally, the dataset is highly consistent, with reliable dates and other variables such as estimated owners, metacritic score, user score, and peak CCU. This consistency ensures that no data is misinterpreted. Only three columns required adjustment: *Supported languages*, *Full audio languages*, and *Estimated owners*. The first two columns contained lists stored as strings ("[language, language]"), so the square brackets were removed and the strings were converted into lists separated by commas. The Estimated owners column contained ranges in the "min - max" format, which were converted into integers before analysis.

2.3 Selection of Data Subsets

To ensure the dataset's quality, relevance, and suitability for information retrieval, several filtering criteria were applied to the initial game collection. These criteria aim to preserve meaningful textual content, maintain temporal relevance, and guarantee the completeness of key descriptive attributes.

First, only games containing more than 100 words in the *About the Game* field were retained. This condition ensures that each document includes a sufficiently rich textual description to support effective indexing, term weighting, and retrieval operations.

Second, only games released within the last 25 years were selected. This constraint focuses the dataset on contemporary titles, improving the relevance of the collection to modern gaming trends and current user interests. Older titles were excluded to reduce noise and align with realistic user search intents in the current gaming context.

Finally, all entries with missing or empty values in the *Categories*, *Tags*, or *Genres* fields were removed. These fields are essential for the organization and faceted navigation of the retrieval system. Ensuring their completeness facilitates both query filtering and the creation of accurate metadata-based subsets in later stages.

2.4 Search Scenarios

To illustrate how users may interact with the information retrieval system, several **search scenarios** were defined based on key metadata fields. Each scenario represents a realistic user query that the final system should support.

- **Age Rating:** A parent searches for games suitable for children under 12 to ensure age-appropriate content.
- **Recent Games:** A gamer looks for titles released in the last five years to discover new or trending releases.
- **Price Range:** A player searches for free-to-play or budget-friendly games under \$10.
- **Languages:** A non-English speaker filters games available in Portuguese for a better gameplay experience.
- **Operating Systems:** A Mac user searches for games fully compatible with macOS.
- **Average Playtime:** A player looks for long, story-driven games with over 50 hours of gameplay.
- **Categories:** A gamer looks for online multiplayer games to play with friends.
- **Genres:** A fan of brain-teasers browses for puzzle games with creative level designs.
- **Tags:** A player searches for indie pixel art games with nostalgic 2D visuals.
- **Developers:** An aspiring developer searches for games made by "ConcernedApe" to study their design style.
- **Publishers:** A user searches for games published by "Devolver Digital" to explore their unique catalog.

These scenarios define the **core information needs** that are expected in the information search system.

3 Exploratory Data Analysis

The exploratory data analysis aimed to obtain an initial understanding of the dataset's structure, content, and overall quality. The available attributes were examined, missing or inconsistent values were identified, and features relevant to the system's goals were assessed. This process helped define which data should be retained for information retrieval development. For the explanation of each columns see Table 1.

3.1 Data Cleaning and Feature Selection

Throughout the data analysis of the file, some columns were identified as irrelevant. At first glance, the first column, *AppID*, was removed since it only represents a unique identifier for each game and does not contribute to the information retrieval process. Similarly, the columns of URLs (*Website*, *Support url*, *Metacritic url*, and *Support email*) were also excluded, as they do not contain textual or numerical information relevant for search or analysis.

With a deeper study of the dataset, some other numerical features were determined to be unrelated and insignificant, as they did not provide meaningful information for the analysis and retrieval tasks.

The columns *Score rank* and *User score* are discarded, as they contained a large number of missing values and provided information already covered by the *Metacritic score*, which offers a more standardized and credible assessment of game quality (Table 2).

The features *Recommendations* and *Discount* were excluded due to their limited relevance to the objectives. *Recommendations* merely quantifies user endorsements, offering little analytical depth, while *Discount* reflects dynamic market conditions unrelated to the informational content of the games.

The columns *Average playtime two weeks* and *Median playtime two weeks* were removed, as they represent short-term player activity that may fluctuate significantly over time. Since the dataset is not continuously updated and reflects data from several months ago, these values would not accurately represent current user engagement.

Lastly, the *Header image*, *Screenshots*, and *Movies* columns were removed, as they contained only links related to their specific media. Since a text-based search system is unlikely to utilize such links, these variables were considered irrelevant.

With these eliminations, the dataset went to 26 columns in total and it was ready for the next stage.

3.2 Data Characterization

After cleaning and feature selection, the dataset was reduced to columns that only provide meaningful insights into the games' attributes. The remaining features can be divided into numerical, textual, and categorical data. Numerical features, such as *Price*, *Estimated owners*, *Peak CCU*, *Positive*, and *Negative* offer quantifiable measures of popularity, commercial performance, and user approval. Textual columns, including *Name*, *About the game*, *Genres* provide descriptive information about each game and are essential for content-based analysis. Finally, categorical features like *Supported languages* and *Developers* allow grouping and filtering games based on common characteristics.

In order to better understand the distribution and relationships among the selected features, a series of visual analyses were built to identify trends and patterns within the dataset.

The *Metacritic score* column has a lot of 0 values. Analysis with Python libraries revealed, before removing the *Metacritic url* column, that whenever its field was empty, the score was recorded as zero. This means that the value 0 does not represent an actual rating, but rather the absence of a Metacritic entry for that game.

The two long textual features, *About the game* and *Notes*, are essential. As the Table 3 shows, the column *About the game* only has 6000 entries without words, while almost 90% of the total entries have no *Notes*. It is also clear that the average word count in *About the game* is significantly higher than in *Notes*. The price distribution, at the Table 4, shows that most games are relatively inexpensive, with a median price of 3.99\$, which is well below the mean of 7.06\$.

The analysis discovered that almost 9% of the total games have more than one developer and, as shown in Figure 2, there are several developers that have been involved in over one hundred games, with a predominance of *EroticGamesClub*. On the other hand, the dataset only has 5.5% of games with multiple publishers, as the most dominant publisher is *Big Fish Games* with more than 500 games published (Figure 3).

Almost every game belongs to the category *Single-player* and 70% of the total games are indie games (Figure 4 and Figure 5).

The Figure 6 illustrates the number of games released each year. It is evident that there has been a crescent evolution in game production over time. It is important to note that the data was last updated on 26th of April, 2025, which explains the low number of released games in 2025. Additionally, the Figure 7 reveals that the average price between 1997 and 2004 was inconsistent, likely due to the low number of games. However, as the years progressed and the technology advanced, the average price tended to slightly decrease.

The feature *Reviews* is a string that usually has a text, a score, and the reviewer in sequence (e.g., "*The Game is really good 10/10 - User1 The Game is really bad 1/10 - User2*"). However, Steam allows users to review without a score. As the Table 5 shows, most of the games do not have reviews. Among those that do, games can receive multiple reviews, and approximately 66% of them do not include a score.

Since the platforms are not inherently compatible, developers must choose which ones to support. The Figure 8 reveals that Windows is the dominant platform, with nearly all games supporting it. The relatively low number of multi-platform games further highlights Windows' supremacy within the Steam ecosystem.

Figure 9 displays the frequency distribution of words extracted from the 'About the Game' column, which contains the most substantial textual content. The analysis reveals that the term "game" appears more than 200.000 times, approximately twice the total number of games in the dataset, indicating its predominant usage in game descriptions.

4 Data Processing Pipeline

Figure 1 illustrates a structured and reproducible data processing pipeline for the Steam dataset. It begins with data collection, followed by preparation and exploratory analysis, then moves to conceptual modeling, and finally preparation for information retrieval.

5 The Conceptual Model

After the Exploratory Data Analysis, the next step was to create a conceptual model of the dataset. If the model were created solely from the available file, it would only contain the *Game* table. However, separating the data into additional tables makes it easier to interpret and more efficient for practical use. The structure can also be analyzed from Steam’s perspective, even without full knowledge of its data model.

In the **Figure 10**, starting with the *Publisher* and *Developer* tables, these entities were appropriate to create because they represent dynamic groups. When a new publisher or developer appears that is not present in the database, it must be added. These entities can also have values or connections with other tables, such as the number of followers, which could link to user information, data that is not included in the current dataset.

It is also important to create tables that support searching and filtering, such as *Categories*, *Genres*, and *Languages*. Although these tables should not frequently have new entries, linking a category to multiple games makes searching more efficient. This allows games to be retrieved based on their association with a specific category, instead of checking every game individually. Since Steam’s database structure is unknown, supported languages for interface and audio were separated into distinct tables. Even though audio language cannot be used as a search filter in Steam’s interface, separating them ensures accuracy, as a language that includes audio does not necessarily provide an interface translation. Therefore, having more than one variable for language support is necessary.

The *Reviews* table was designed separately, as the dataset’s reviews are external and independent from Steam’s internal review system. Each review includes multiple data points, such as description, evaluation, and source website, which would make comprehension difficult if all reviews were stored within the same record.

Tags were treated as another independent table. These are created freely by publishers and are not standardized or searchable in the same way as categories or genres. However, since many tags are reused, storing them in a separate table simplifies creation, improves efficiency, and supports recommendation systems.

Other potential tables, such as those representing *Positive* and *Negative* review values, were not included. Because the dataset lacks review text and user identifiers, creating a table or boolean variable to represent review sentiment would

provide little analytical value, as the counts of positive and negative reviews would need to be recalculated each time.

If the dataset contained *Owners* instead of *Estimated Owners*, it would have been possible to establish simpler connections between Users (or user libraries) and *Games*, resulting in a more streamlined database structure.

The remaining values, such as *Name* and *Release Date*, are attributes specific to each game and, therefore, remain within the *Game* table.

6 Follow-Up Information Needs

After characterizing the dataset, it becomes necessary to define a set of follow-up information needs that reflect the types of searches users are likely to perform within the Steam games domain. These needs emerge naturally from the metadata available in the collection and provide the conceptual basis for the retrieval tasks.

A central group of information needs concerns the discovery of games with particular themes, genres or gameplay mechanics, ranging from horror and retro titles to puzzle games, survival experiences or multiplayer action. These rely on descriptive metadata such as genres, tags, categories and the about-the-game text. Other needs relate to audience suitability, where users seek age-appropriate or casual versus hardcore games, drawing on fields such as age ratings and playtime labels.

Technical and accessibility needs also shape user queries, including requirements for specific operating systems, interface or audio languages and other platform constraints. Commercial and temporal needs further influence search behaviour, with users looking for expensive or low-cost games, recent releases or older classics. Finally, production-oriented needs emerge when users search for titles developed or published by specific companies, informed by the developers and publishers fields.

Together, these information needs capture the breadth of realistic user behaviour.

7 Dataset Indexing: Processing and Solr Schema

This section details the data processing workflow. We discuss the dataset’s initial structure, the transformations applied to enable faceted search, and the definition of the Solr schema and field types used for indexing

7.1 Dataset Characteristics

The original dataset was in CSV format. To facilitate indexing and customized search in Solr, the CSV was converted into JSON.

This JSON representation enables Solr to handle textual, categorical, numeric, and multi-valued fields efficiently, forming the basis for a personalized Solr index that supports free-text search, filtering, and faceting.

The dataset contains a variety of data types, including arrays for multi-valued attributes and booleans for platform compatibility. The specific fields present in the JSON structure are detailed in Table 6.

To reduce load on the Solr container, a subset of 20,000 randomly selected documents from the dataset was chosen for indexing.

7.2 Data Transformations

Prior to indexing, several transformations were applied to prepare the dataset for robust search and faceted exploration.

Age-related information was also standardized. Games without explicit restrictions were labeled as “All Ages,” while additional labels were introduced to reflect suitability levels: “Kids” (-13), “Teen” (13+), “Mature” (17+), and “Adult” (18+). These values were stored in a newly added column, *age_rating_label*.

Similarly, games were categorized by release year into “Old” (2000–2010), “Moderately Old” (2011–2015), “Recent” (2016–2020), and “Very Recent” (2021–2025), and this classification was stored in the *release_year_label* column.

Pricing information was grouped into descriptive categories: “Free” for games priced at 0 euros, “Cheap” for prices below 5 euros, “Affordable” for the range between 10 and 25 euros, and “Expensive” for prices above 25 euros. The resulting labels were recorded in the *price_range_label* column.

Operating system compatibility for Windows, Linux, and Mac was consolidated into labels such as “Windows only,” “Windows + Linux,” or “Windows + Mac,” based on the corresponding boolean fields, which were removed after transformation. The generated compatibility descriptions were stored in the *OS_label* column.

Finally, average playtime was grouped into the categories “Casual” (less than 10 hours), “Moderate” (10–50 hours), and “Hardcore” (more than 50 hours). These classifications were captured in a *playtime_label* column.

These transformations ensured that both textual and categorical fields were consistently structured and optimized for indexing, enabling efficient search and faceted navigation.

7.3 Solr Schema and Field Types

Following the transformations, the dataset was indexed into Solr according to a custom schema designed for free-text search and structured filtering.

- **text_general_game:** The analyzer includes StandardTokenizerFactory to split text into tokens, ASCIIFoldingFilterFactory to convert accented or non-ASCII characters to ASCII, LowerCaseFilterFactory for lower-case normalization, StopFilterFactory to remove common English stopwords, and SnowballPorterFilterFactory (English) for stemming.

- **knn_vector_384:** Defined as a `solr.DenseVectorField` with a 384-dimensional embedding vector and cosine similarity.
- **float_field / int_field:** Used for numeric fields enabling range queries, sorting, and boosting.
- **date_field:** Implements `DatePointField` to support range queries, sorting, and date-based filtering.

This integrated processing and indexing strategy ensures a robust, efficient, and user-friendly search system combining free-text retrieval, structured filtering, and evaluation of retrieval effectiveness, the indexed fields are detailed in Table 6.

8 Retrieval Process

After the dataset was indexed in Solr, the next objective was to implement a retrieval process capable of satisfying the information needs previously defined in the search scenarios. This involved designing effective queries, choosing appropriate fields for retrieval, configuring scoring mechanisms, and defining two retrieval setups for the evaluation.

8.1 Information Needs and Query Interpretation

Information Needs are typically expressed in natural language: short, often ambiguous, and lacking explicit field constraints. To interpret such queries, the system relies on Solr’s **Extended DisMax (eDisMax)** parser, which supports multi-field retrieval with field-specific boosting, fuzzy and wildcard matching, and phrase or proximity constraints. This enables the system to treat user queries as semantically structured units and distribute terms across relevant fields according to domain knowledge.

Four representative information needs are defined below as user queries, described in terms of their underlying information needs.

Query 1. Hardcore French games for Linux. challenging games available in French for the Linux platform. The focus is on a high difficulty level with above average playtime, language compatibility, and OS Support.

Query 2. Cheap adventure games for Windows. The user is looking for adventure-genre titles that run on Windows and fall within a low price range. The emphasis is on filtering by both platform and genre while prioritizing affordability.

Query 3. Casual Single-Player Zombie Games. This query targets games that are easygoing and designed for solo play, with a focus on zombie-themed content.

Query 4. Recent and expensive games from Ubisoft. The user is looking for newly released games developed or published by Ubisoft, emphasizing recent titles on the higher end of the price spectrum.

8.2 Field Selection

The eDisMax query parser requires an explicit selection of fields for each query so that the retrieval process focuses on the attributes most relevant to the user's intent. The field mappings for our queries were designed as follows:

Query 1: Fields *OS_label* (Linux compatibility), *supported_languages* and *full_audio_languages* (French support), and *playtime_label* (Hardcore classification).

Query 2: Fields *price_range_label* (Cheap/Budget), *OS_label* (Windows), and *about_the_game* combined with *tags* (Adventure genre).

Query 3: Fields *about_the_game* (capture relevant terms in description), *categories* (Single-player), *playtime_label* (Casual), and *title* (Zombie keyword in Title).

Query 4: Fields *developers* and *publishers* (Ubisoft), *release_year_label* (Recent), and *price_range_label* (Expensive).

Together, these field selections enable eDisMax to interpret and operationalize each information need in a structured and semantically meaningful way.

8.3 Demonstration

To experiment with these information needs, it is possible to create a query by filling the fields on the Query page at Solr. At this page the relevant query fields are "q" (query) that focuses on the most valuable words in the query, "q.op" (query operator) which utilizes OR | AND for query operations, "fq" (filter query) that defines a query that can be used to restrict the superset documents and "defType" (query parser) to specify which query parser to use. After selecting *edismax*, the "qf" field specifies which fields to search and their relative importance. For each one, the steps required to reproduce the query in Solr's Query page are provided.

To reproduce the first query on Solr's Query page:

1. q: "Hardcore french game that can be played on Linux";
2. *defType: "edismax";
3. qf: "OS_label supported_languages playtime_label full_audio_languages".

After executing the query, the result will be the same as the one shown in **Figure 11**

To reproduce the second query on Solr's Query page:

1. q: "Cheap adventure games for Windows";
2. defType: "edismax";
3. qf: "price_range_label OS_label about_the_game tags".

After executing the query, the result will be the same as the one shown in **Figure 12**

To reproduce the third query on Solr's Query page:

1. q: "Casual single player zombie games";
2. defType: "edismax";
3. qf: "about_the_game categories playtime_label title".

After executing the query, the result will be the same as the one shown in **Figure 13**

To reproduce the last and forth query on Solr's Query page:

1. q: "recent and expensive games from Ubisoft";
2. defType: "edismax";
3. qf: "developers publishers release_year_label price_range_label".

After executing the query, the result will be the same as the one shown in **Figure 14**

9 Evaluation

The evaluation focuses on assessing how effectively the retrieval system satisfies the information needs outlined in the search scenarios. This involves executing the predefined queries, analysing the relevance of the returned results, and comparing the performance of two retrieval setups. The goal is to measure the system's ability to retrieve accurate, meaningful, and contextually appropriate documents, thereby determining the overall quality and robustness of the implemented search approach.

9.1 Boosting and Filtering Setups for Evaluation

The boosting and filtering mechanisms offered by eDisMax enable the retrieval process to highlight the most informative attributes while constraining the result set to match explicit user requirements. Filtering imposes strict conditions that a document must satisfy to be considered at all, whereas boosting adjusts ranking weights to promote documents matching important fields or phrases. Together, these techniques refine the search space and strengthen the system's ability to interpret queries in a manner consistent with the user's underlying information need.

For the query "*Hardcore French games for Linux*", filtering required Linux compatibility, French language availability, and a hardcore playtime classification, applied through *OS_label*, *supported_languages*, and *playtime_label*, respectively. Field-level boosting emphasized these same attributes, with *OS_label* receiving a strong weight (boost ^4), *supported_languages* a moderate weight (boost ^3), and *playtime_label* a smaller but relevant weight (boost ^2), promoting documents that closely matched the requested constraints (**Figure 15**).

In the query "*Cheap adventure games for Windows*", filtering was applied on *OS_label* to restrict results to Windows-compatible games. Boosting focused on affordability and genre, with *price_range_label* receiving the highest field-level weight (boost ^4) to prioritize low-cost titles, while *about_the_game* and *tags* were moderately boosted (boost ^3) to promote adventure-related content. A custom term-frequency boost further emphasized documents where the term "adventure" appeared prominently within descriptions or metadata (**Figure 16**).

For "*Casual single player zombie games*", no filtering constraints were applied, allowing broader coverage. Ranking

was guided through boosting of several fields: *categories* was strongly weighted (boost ^4) to highlight gameplay modes, while *about_the_game* and *playtime_label* were boosted moderately (boost ^3) to favor zombie-related and casual titles. Phrase-level boosting reinforced relevance through the proximity of the phrase “zombie singleplayer” (proximity ~10, boost ^6), and additional term-frequency boost rewarded documents containing both the terms “zombie” and “multi”, using a wildcard “*” on “single” to capture variations such as “singleplayer” and “single-player”.

Finally, for “*Recent and expensive games from Ubisoft*”, filtering on *release_year_label* and *price_range_label* ensured that only recent and high-cost games were considered. Field-level boosting reinforced these constraints, with both *release_year_label* and *price_range_label* receiving strong weights (boost ^5). Additional boosts on *developers* (boost ^2) and *publishers* (boost ^3), combined with term-frequency boosting for occurrences of “Ubisoft,” increased the ranking of titles explicitly referencing the publisher (Figure 17).

Across all queries, filtering served to enforce essential constraints such as platform, language, price category, or recency, while field boosting modulated ranking to highlight documents whose content matched the most informative attributes of the user request. Phrase-level and term-frequency boosting further refined ranking by emphasizing both term proximity and prominence, allowing the retrieval system to move beyond surface-level text matching toward a ranking structure that more accurately reflects the deeper semantic intent of each query (Figure 18).

9.2 Construction of Qrels for Evaluation

To evaluate the effectiveness of the retrieval system, relevance judgments must be established for each information need in the form of qrels. As the dataset lacks pre-existing relevance annotations, these judgments must be inferred manually based on the expected behaviour of each query. For every information need, a set of representative documents is sampled from the collection and assessed as relevant or non-relevant according to whether they satisfy the semantic and structural constraints encoded in the query.

Qrels were constructed using a systematic pooling methodology. Each query was executed on Solr with multiple combinations of boosting configurations, and for each run, the top 10 documents that best represented the query’s objectives were manually identified and recorded. This process was repeated over several iterations to ensure comprehensive coverage and capture the most relevant candidate documents.

For the query “*Hardcore French games for Linux*”, documents were considered relevant if the games supported Linux as an operating system, included French language support for both text and audio, and offered more than 50 hours of gameplay. The most relevant results were those that satisfied all these criteria. Among them, operating system support was

considered the most important characteristic, since Linux is not a commonly supported platform for games.

For the query “*Cheap adventure games for Windows*”, price was the primary relevance factor, as affordability was the main constraint. Adventure games were the second most important characteristic. Since approximately 99% of games support Windows, operating system compatibility was not considered a discriminating factor.

For the query “*Casual single player zombie games*”, relevance was mainly determined by the game category (single-player) and genre (zombie). Playtime was considered less important and did not significantly influence the selection of relevant documents.

For the query “*Recent and expensive games from Ubisoft*”, relevant documents were identified as the most recent and high-profile titles developed or published by Ubisoft. Developer/publisher information and release date were the most important characteristics, while the price constraint was considered less critical.

After completing all iterations, the 10 documents with the highest frequency of appearance across all runs were selected for inclusion in the qrels, with the document appearing most frequently assigned the highest relevance ranking. This approach ensured that the resulting qrels reflected both consistency across different boosting strategies and the documents’ alignment with the intended query semantics.

The final phase of the evaluation involves quantifying and analysing retrieval performance, including assessing the impact of the implemented boosting techniques on ranking effectiveness.

9.3 Evaluation Setup

To assess the effectiveness of the retrieval system, each user query was executed under two conditions: first, as a baseline with no boosting or filtering applied, and second, with the boosting and filtering configurations described in Section 9.1. The relevance of retrieved documents was measured against the qrels constructed as outlined in Section 9.2, ensuring that evaluation reflected the semantic and structural requirements of each query.

Retrieval results for both the baseline and boosted queries were collected and analysed using standard information-retrieval metrics. This setup allowed a direct comparison between the performance of unmodified queries and those enhanced with boosting and filtering, providing a clear view of the impact of these techniques on ranking quality.

9.4 Results

To investigate the improvements made by boosting, we utilized *trec_eval* [4] to compute quantitative performance metrics. The evaluation compares the baseline query (Q1 - Simple) against the boosted configuration (Q2 - Boosted) using three standard metrics: Average Precision (AP), Area Under the Curve (AUC), and Precision at 10 (P@10). Each of these

metrics captures a different aspect of the effectiveness of the ranking produced by the query.

The following discussion analyzes the results for each information need sequentially, referencing the Precision-Recall curves and the metric values summarized in Table 8 and in Table 9.

1. Hardcore French games for Linux. For the first information need (Figure 19), illustrates that the boosted query (Q2) performs consistently better than the simple query (Q1). As shown in Table 8, the boosted configuration resulted in an increase of approximately 0.2 units across all three metrics. While the improvement is notable, the baseline query already provided relatively strong results, meaning the boosting refined an already decent retrieval rather than salvaging a failed one.

From the detailed retrieval statistics in Table 9, both setups retrieve 10 documents (Num Ret = 10). The boosted setup retrieves more relevant documents (R increases from 1 to 5) and fewer non-relevant ones (NR decreases from 9 to 5). The reciprocal rank (RR) also improves from 0.2000 to 1.0000, meaning that under Q2 the first relevant document appears at rank 1.

2. Cheap adventure games for Windows. In contrast to the first scenario, the simple query for this information need produced very poor results (Figure 20), remaining below 0.1 across most metrics. However, the boosted query demonstrated a drastic improvement, becoming more than 25 times more precise and achieving an AUC of 0.6. This performance gap is likely due to Steam (dataset domain) containing a massive volume of cheap Windows games. A simple query gets lost in this noise, returning irrelevant cheap items. The boosted query, by emphasizing key fields and terms, successfully filtered the noise to match relevant documents effectively.

Table 9 shows that both Q1 and Q2 retrieve 10 documents, and Q2 retrieves slightly more relevant documents overall (R increases from 6 to 7), while decreasing the number of non-relevant results (NR decreases from 4 to 3). In this case, RR remains 1.0000 in both setups, meaning that the top-ranked result is relevant in both runs. The key difference is therefore not in the first hit, but in the overall ranking quality reflected in AP/AUC/P@10.

3. Casual Single-Player Zombie Games. For the third information need, the results (Figure 21) also show big performance improvement by the boosted query. As detailed in the Table 8, the boosted query resulted in improvement across the board, with a specific increase of 0.5 for Precision at 10 (P@10). This specific metric increase suggests that the boosting mechanism was highly effective at pushing relevant zombie titles to the very top of the search results, directly improving the user's immediate experience.

The retrieval statistics in Table 9 confirm this trend: while both runs retrieve 10 documents, Q2 retrieves more relevant documents overall (R increases from 2 to 7) and fewer non-relevant ones (NR decreases from 8 to 3). In this query, RR remains unchanged at 0.5000, suggesting that the first relevant result appears at rank 2 for both runs, but the overall relevance density differs substantially between the two runs.

4. Recent and expensive games from Ubisoft. Finally, the graph for the fourth information need (Figure 22) presents a unique case where the boosted query achieved a perfect score across all metrics. This outcome is likely due to the low document frequency of Ubisoft games in this specific sample, which made it easier for the boosted query to successfully retrieve every single relevant document. In fact, under these circumstances even the simple query performed exceptionally well, retrieving most of the relevant documents as we can see in the last row of Table 8.

In Table 9, Q1 retrieves fewer relevant documents (R = 8) and includes non-relevant results (NR = 2). In contrast, Q2 retrieves all relevant documents (R = 10) with no non-relevant results (NR = 0). RR is 1.0000 for both runs, indicating that a relevant Ubisoft title appears immediately at rank 1 in both setups.

10 Semantic Search

The semantic search functionality in our system is implemented outside Solr using pre-computed sentence embeddings. A SemanticSearchEngine based on the *sentence-transformers* model all-MiniLM-L6-v2 is lazily initialized when the first semantic request is made, avoiding unnecessary load when users only perform keyword searches.

For each game, an embedding vector is generated offline and stored in a JSON file (embeddings/game_embeddings.json) together with the corresponding game identifiers. At query time, the Flask backend encodes the user query into an embedding and retrieves the top-*k* most similar games using cosine similarity. This produces an ordered list of candidate game ids.

Because the embedding file only stores vectors and ids, the backend then retrieves the full document metadata from Solr by issuing an OR query over the selected ids (e.g., id: "... " OR id: "... "). The returned documents are re-ordered according to the semantic similarity ranking, and each result is enriched with a semantic_score field, exposing the similarity value used for ranking.

This design separates semantic retrieval from Solr indexing while still leveraging Solr as the authoritative store for the full game metadata shown in the interface.

11 Hybrid System

A **hybrid retrieval system** combines keyword-based search (Solr/eDisMax) with semantic search (embedding similarity).

In our implementation, the hybrid mode executes both components and merges their outputs at the application layer.

First, a keyword query is issued to Solr using eDisMax over the main descriptive and categorical fields (e.g., *title*, *about_the_game*, *genres*, *categories*, *tags*, and label fields). In parallel, the semantic engine computes the query embedding and returns a similarity-ranked list of candidate games. To increase overlap and improve reranking stability, the system initially fetches more candidates than requested (twice the target number of rows) from each component.

The two result sets are then unified by document id. Keyword scores are normalized by dividing each Solr score by the maximum score observed in the keyword candidate set. Semantic similarity scores are taken from the semantic retrieval stage. For each document d and query q , the final hybrid score is computed by a weighted combination implemented in the semantic engine:

$$s_{hyb}(d, q) = \alpha s_{kw}(d, q) + (1 - \alpha) s_{sem}(d, q),$$

where s_{kw} is the normalized Solr score, s_{sem} is the cosine similarity, and α (keyword_weight) controls the trade-off between exact matching and semantic matching (default $\alpha = 0.5$). The merged list is then sorted by hybrid_score, and each returned document includes hybrid_score, keyword_score, and semantic_score to support transparency.

This approach preserves the advantages of Solr’s lexical matching and structured fields while improving robustness to vocabulary mismatch through semantic similarity.

12 New System Evaluation

This section extends the evaluation to the **new retrieval system**, comparing four retrieval strategies: *Normal* (simple eDisMax query), *Boosted* (eDisMax with boosting/filtering as in Section 9.1), *Semantic* (embedding-based cosine similarity), and *Hybrid* (weighted combination of keyword and semantic scores as described in the Hybrid System section).

For consistency across systems, the original information needs were kept, but the user queries were simplified to short keyword strings, removing filler words that could introduce noise. The resulting queries are: Query 1: *hardcore french linux*; Query 2: *cheap adventure windows*; Query 3: *casual single player zombie*; and Query 4: *recent expensive ubisoft*.

All runs were evaluated with *trec_eval* using the same grels described in Section 9.2. For each query, we retrieved the top 10 documents and computed summary statistics. We report the same **retrieval diagnostics** used in Table 9: Num Ret (retrieved documents), R (relevant retrieved), NR (non-relevant retrieved), and RR (reciprocal rank of the first relevant document).

12.1 Precision–Recall Curves

The Precision–Recall curves for the four retrieval approaches are shown in **Figure 25** through **Figure 28**. These plots provide a visual comparison of ranking quality under each

strategy for the four simplified queries. Table 11 summarizes the *trec_eval* diagnostics for all queries and retrieval modes.

12.2 Discussion

Overall, the results show that the **Boosted** keyword setup remains the most reliable approach across all four information needs, achieving the highest number of relevant documents in the top 10 for every query.

For Query 1 (hardcore/french/linux), boosting improves early precision substantially (RR improves from 0.2 to 1.0) and doubles the number of relevant retrieved documents. In contrast, the semantic and hybrid configurations fail to retrieve relevant documents in the top 10, suggesting that the embedding representation and/or candidate selection is not capturing the strict constraints implied by platform and language.

For Query 2 (cheap/adventure/windows), boosting produces a strong improvement in both relevance coverage ($R=9$) and ranking ($RR=1.0$). The semantic approach retrieves only one relevant item, and the hybrid run returns no relevant results in the top 10, indicating that score fusion did not help for this query and that the semantic candidates may have diluted the lexical constraints around price and platform.

For Query 3 (casual/single player/zombie), both normal and boosted keyword runs retrieve relevant documents, but boosting increases relevant density in the top 10 (R from 5 to 9). Semantic and hybrid retrieve fewer relevant documents, although the hybrid configuration achieves $RR=1.0$, meaning it places a relevant document at rank 1, even if overall coverage remains low.

For Query 4 (recent/expensive/ubisoft), both normal and boosted keyword runs perform strongly, with boosting achieving perfect relevance in the top 10. Semantic retrieval fails to retrieve relevant results for this query, which is expected when the information need is dominated by a named entity (*Ubisoft*) and structured labels (recency/price), while the hybrid approach partially recovers performance ($R=7$) but remains below the purely boosted keyword baseline.

12.3 Semantic Search Performance Analysis

The evaluation results in Table 11 show that semantic search significantly underperforms keyword-based approaches, with particularly poor results for Queries 1, 2, and 4 where no relevant documents were retrieved in the top 10 results. While this may initially appear as a fundamental failure of the semantic approach, several characteristics of our evaluation setup explain these results and provide important insights into when semantic search is appropriate.

12.3.1 Dataset and Query Characteristics. The evaluation framework operates under conditions of extreme relevance sparsity. From a collection of 20,000 indexed documents, only 10 documents were deemed relevant for each information need. This represents a relevance rate of just

0.05% (10 out of 20,000). In such conditions, even small misalignments between the query embedding and document embeddings can easily push relevant documents outside the top-10 retrieval set.

Keyword-based boosting with explicit field weighting succeeds precisely because it enforces hard constraints such as `fq: OS_label:Linux` that immediately reduce the candidate pool to hundreds rather than thousands of documents. Semantic search, by contrast, ranks documents purely by embedding similarity without applying these constraints, meaning relevant documents must compete against the entire 20,000-document collection based solely on semantic closeness.

Our evaluation queries are intentionally structured as multi-constraint filters rather than natural language descriptions. Query 1 requires simultaneous satisfaction of three independent categorical constraints: playtime level, language support, and operating system. Query 2 combines price range, genre, and platform requirements. Query 4 specifies publisher, recency, and price tier. These queries are fundamentally non-deterministic in the sense that there is no single semantic concept they represent. Instead, they define a Boolean intersection of independent metadata conditions.

Semantic embeddings, which are designed to capture holistic meaning and context, struggle with such queries for several reasons. First, the embedding model treats terms like *hardcore*, *french*, and *linux* as semantically related terms rather than as three separate required filters. A document about a hardcore game mentioning French history may rank highly despite not supporting the French language or Linux platform. Second, our game embeddings concatenate descriptive text from *about_the_game* with categorical labels like *OS_label* and *playtime_label*. The descriptive text dominates the embedding space, meaning that semantic similarity is driven primarily by gameplay descriptions rather than the structured metadata that our queries actually target. Third, short keyword-style queries produce embeddings with limited context. The embedding for *hardcore french linux* captures an abstract concept that may align poorly with the verbose game descriptions in the index, even when those descriptions contain the same terms.

12.3.2 Where Semantic Search Would Excel. It is important to note that semantic search is not fundamentally flawed but rather poorly suited to the specific query types and evaluation constraints. Semantic retrieval would be expected to perform well for natural language queries such as *relaxing games where I explore beautiful landscapes*, which were not evaluated. Such queries benefit from semantic understanding because they describe desired experiences rather than categorical requirements. Similarly, synonym-rich queries like *racing games* would benefit from semantic

search returning results about driving, motorsport, and formula one without requiring explicit keyword matches. Underspecified queries where users provide vague descriptions allow semantic search to identify relevant documents that keyword matching would miss.

12.3.3 Hybrid System Performance. The hybrid system described in Section 11 combines normalized keyword scores with semantic similarity scores using a weighted sum with default parameter α equal to 0.5. However, in our evaluation, hybrid retrieval frequently performs worse than either pure approach. For Query 2, the hybrid system achieved zero relevant documents, worse than both the boosted keyword approach with nine relevant documents and the pure semantic approach with one relevant document. For Query 4, hybrid retrieval found seven relevant documents, falling short of the boosted keyword baseline of ten relevant documents.

This occurs because the hybrid fusion dilutes strong keyword signals with weak semantic signals. When semantic similarity is systematically misaligned with relevance as in our multi-constraint queries, averaging the two scores degrades rather than improves ranking quality. The score normalization step may also introduce artifacts. Solr scores are normalized by dividing by the maximum score in the candidate set, which can distort relative differences when the keyword candidates are heterogeneous in their relevance.

12.3.4 Implications for System Design. The results suggest that a production retrieval system should not use a single fixed approach for all queries. For structured queries containing explicit metadata constraints such as platform specifications, price ranges, publisher names, or language requirements, the system should use boosted keyword search with field filtering. For exploratory or thematic queries that describe desired gameplay experiences or themes using natural language, semantic or hybrid search becomes more appropriate. A simple filter heuristic could detect constraint keywords like *linux*, *cheap*, or *ubisoft* and route the query to the appropriate retrieval backend.

12.3.5 Quantitative Summary. Table 10 summarizes the performance gap between boosted keyword and semantic approaches across all queries. The boosted keyword approach retrieved an average of 8.0 relevant documents compared to 0.75 for semantic search, representing a tenfold performance difference. The average reciprocal rank for boosted keyword was 0.875 versus 0.057 for semantic search, indicating that relevant documents appear much earlier in keyword-based rankings. Most critically, semantic search produced zero relevant results for two out of four queries, representing complete retrieval failures.

Despite this substantial performance gap in our evaluation, it is critical to recognize that this reflects the mismatch between semantic search strengths and our evaluation characteristics. Semantic search excels at conceptual similarity and vocabulary flexibility but struggles with extreme sparsity, multi-constraint Boolean queries, and structured metadata filtering. The results should not be interpreted as evidence that semantic search is universally inferior, but rather that it requires careful application in domains and query contexts where its design assumptions align with user information needs.

12.3.6 New System Conclusions. The poor performance of semantic and hybrid search in this evaluation is primarily attributable to three factors. First, the evaluation design imposed extreme relevance sparsity with only 10 relevant documents out of 20,000 total documents, and the query types favored Boolean filtering over semantic matching. Second, the embedding implementation used concatenated field embeddings that do not respect categorical constraints or metadata structure. Third, there was a fundamental mismatch between short structured queries designed for metadata filtering and the semantic similarity approach optimized for natural language understanding.

These findings emphasize that retrieval system design must consider the expected query distribution and information needs. For the Steam game domain, boosted keyword search with explicit field weighting remains the most effective approach for the types of structured multi-constraint queries evaluated in this study. Semantic search remains a valuable technique for exploratory queries where users describe desired experiences rather than enumerate specific requirements.

13 Graphical User Interface

To enhance usability and facilitate interaction with the information retrieval system, we developed an intuitive and user-friendly graphical user interface (GUI). The GUI serves as the primary point of interaction between users and the system, allowing them to submit queries, explore retrieved results, and refine searches without requiring technical expertise. The interface was designed with usability and clarity as core objectives. Key components—including the query input area, results display section, and filtering controls—are logically organized to minimize cognitive load and improve user efficiency.

The GUI is composed of three main sections: the search area on the main page, the results area, and a dedicated game page for detailed exploration of individual games. In the search area, users can freely enter their queries and choose the type of search to be performed. The system supports keyword-based searches using Solr indexing, semantic searches using AI for meaning-based matching, or a hybrid approach that combines both methods for optimized retrieval.

Users can also select the number of documents to return, ranging from 10 to 50 results. Beneath the search controls, a filtering section allows users to refine results according to price range, release period, age rating, operating system, and average gameplay time, thereby enabling a more personalized search experience.

The results area displays the games retrieved by the system. Each result includes a header image for easy recognition, the game name, and the relevance score assigned by the retrieval system. Results are presented in descending order of score, with the most relevant document appearing first. Additional information for each game includes the release date, price, publishers, a brief summary, and associated tags. For transparency and traceability, the document ID from the Solr system is also displayed.

The game page provides a more detailed view of a selected game. It features a larger version of the header image and the game name, followed by key details such as release date, price, developers, publishers, achievements, and the number of positive and negative reviews. A comprehensive summary of the game is included, alongside all genres, categories, and tags associated with it. Reviews are parsed for clarity, displaying the review text, the user who wrote it, and, when available, a review score out of 100.

14 Conclusion and Future Work

This work presented the transformation of a raw Steam dataset into a functional, evaluated text-based retrieval system. The process encompassed data preparation and analysis, the definition of a conceptual model, and the implementation of a custom indexing schema in Apache Solr. Evaluation using predefined user scenarios demonstrated that carefully designed boosting configurations led to measurable improvements in ranking quality, particularly in metrics such as Average Precision and Precision at 10. Overall, the system proved capable of retrieving relevant games across a range of information needs within the Steam domain.

Further exploration of semantic search could involve several refinements. Field-aware embeddings may be used to encode metadata fields separately from descriptive text, enabling more precise weighting during retrieval. The embedding model could also be fine-tuned on domain-specific game retrieval tasks, particularly those involving multi-constraint queries, to better align with user needs. Alternatively, a two-stage retrieval architecture could be adopted, using semantic search for initial ranking followed by strict filtering on categorical constraints to ensure that required conditions are met.

References

- [1] Coursera. 2025. What is Hugging Face? <https://www.coursera.org/articles/what-is-hugging-face>. Accessed: 2025-10-08.
- [2] FronkonGames. 2025. Profile. <https://huggingface.co/datasets/FronkonGames>. Accessed: 2025-10-08.

Table 1. Description of the 40 dataset variables

Column	Description
AppID	Unique Steam game identifier
Name	Game title
Release date	Date of release on Steam
Estimated owners	Estimated ownership range
Peak CCU	Peak concurrent users
Required age	Minimum recommended player age
Price	Game price in USD
Discount	Current discount percentage
DLC count	Number of downloadable content items
About the game	Full store description
Supported languages	Languages for interface and subtitles
Full audio languages	Languages with full voice-over
Reviews	Reviews of the game
Header image	Store banner image URL
Website	Official game website
Support URL	Technical support webpage
Support email	Technical support contact email
Windows	Availability on Windows platform
Mac	Availability on macOS platform
Linux	Availability on Linux platform
Metacritic score	External critic score (0–100)
Metacritic url	URL to the external critic score
User score	Steam user rating (0–10)
Positive	Count of positive user reviews
Negative	Count of negative user reviews
Score rank	Relative ranking (percentage scale)
Achievements	Number of Steam achievements
Recommendations	Number of user recommendations
Notes	Additional remarks or comments
Average playtime forever	Mean lifetime playtime (minutes)
Average playtime two weeks	Mean playtime in last two weeks (minutes)
Median playtime forever	Median lifetime playtime (minutes)
Median playtime two weeks	Median playtime in last two weeks (minutes)
Developers	Company or individuals who developed the game
Publishers	Company that published the game
Categories	Game modes or features (e.g., single-player)
Genres	Main genre (e.g., action, strategy)
Tags	User-assigned descriptive labels
Screenshots	URLs to in-game screenshots
Movies	URLs to trailers or videos

- [3] IBM. 2025. What is Hugging Face? <https://www.ibm.com/think/topics/hugging-face>. Accessed: 2025-10-08.
- [4] National Institute of Standards and Technology. 2025. National Institute of Standards and Technology. <https://www.nist.gov/>. Accessed: 2025-11-16.
- [5] Apache Solr. 2025. Apache Solr. <https://solr.apache.org/>. Accessed: 2025-10-11.

Table 2. Data Summary for Game Scores

Metric	Count
Invalid Metacritic scores	107447
Invalid user scores	111408
Invalid score ranks	111408
Total records	111452

Table 3. Word Count Statistics for the *About the Game* and *Notes* Columns

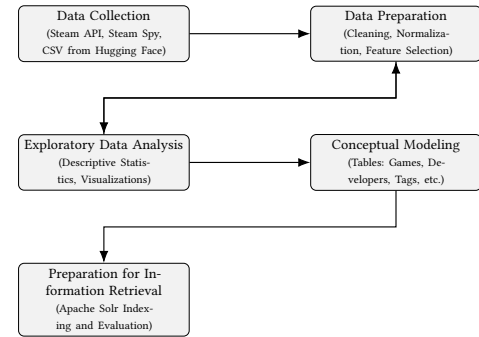
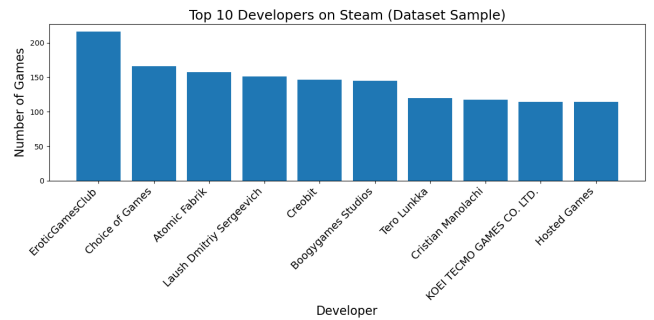
Column	Without Words	Minimum	Maximum	Average	Median
About the Game	6483	1	8402	209.56	172
Notes	93002	1	325	24.35	18

Table 4. Price Statistics for the Steam Games

Statistic	Minimum	Maximum	Average	Median
Price (\$)	0.00	999.98	7.06	3.99

Table 5. Game Reviews Analysis Summary

Metric	Count/Value
Review Coverage	
Games without reviews	101,057
Total reviews	24,638
Review Distribution	
Maximum reviews per game	10
Average reviews per game	0.22
Review Quality	
Reviews with score	7,567
Reviews without score	17,071

**Figure 1.** Structured and reproducible data processing pipeline for the Steam dataset.**Figure 2.** Top 10 Developers on Steam

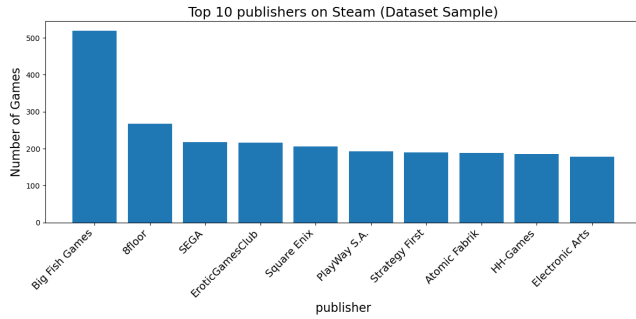


Figure 3. Top 10 Publishers on Steam

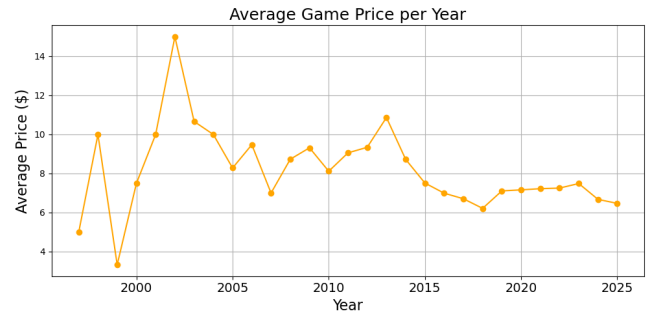


Figure 7. Average Price Per Year

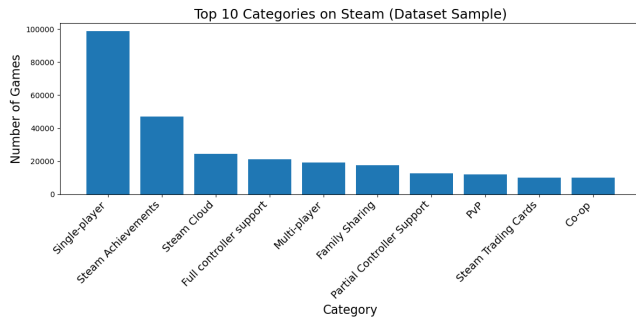


Figure 4. Top 10 Categories on Steam

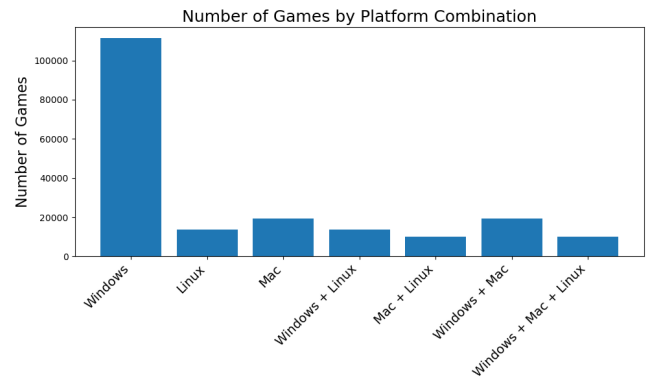


Figure 8. Number of Games by Platform Combination

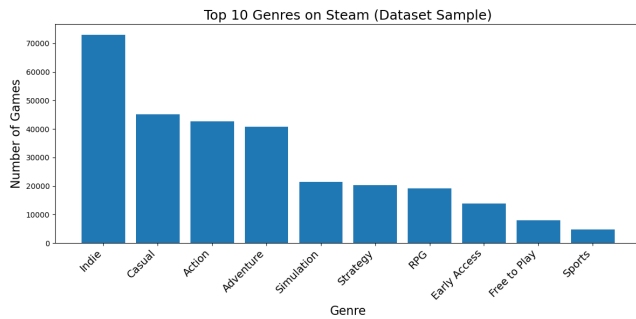


Figure 5. Top 10 Genres on Steam

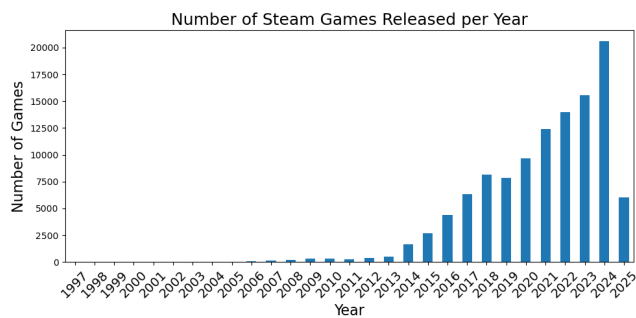


Figure 6. Number of games per year on Steam

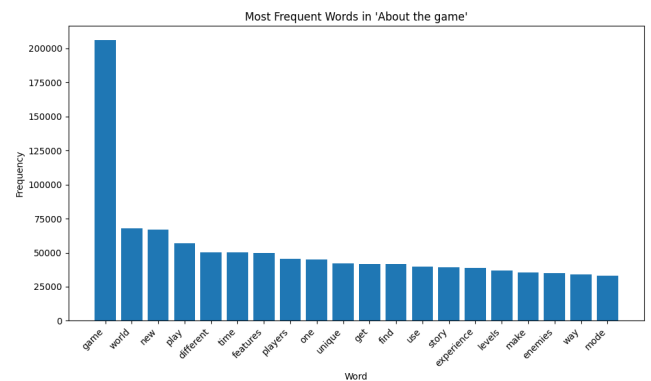


Figure 9. Most frequent words in 'About the Game' column

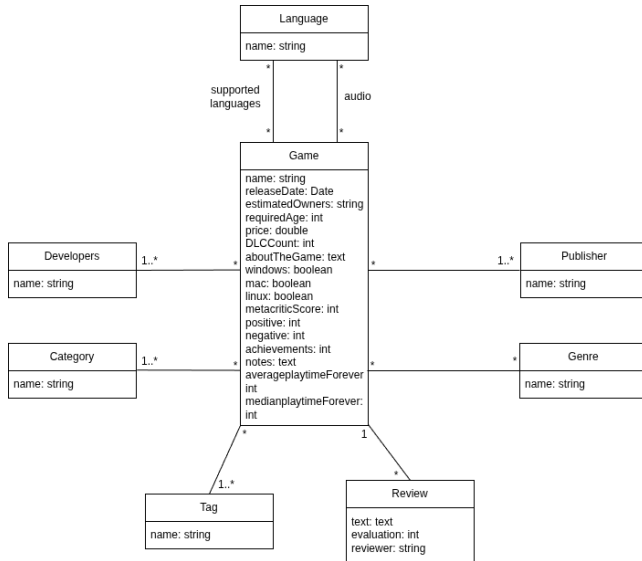


Figure 10. Conceptual model

Table 6. Fields present in the JSON dataset.

Field Name	Data Type
<i>id</i>	Unique Identifier
<i>title</i>	Text
<i>release_date</i>	Date
<i>estimated_owners</i>	Array
<i>peak_CCU</i>	Integer
<i>required_age</i>	Integer
<i>price</i>	Float
<i>DLC_count</i>	Integer
<i>about_the_game</i>	Text
<i>supported_languages</i>	Array
<i>full_audio_languages</i>	Array
<i>reviews</i>	Text
<i>windows, mac, linux</i>	Boolean
<i>positive, negative</i>	Integer
<i>achievements</i>	Array
<i>average_playtime_forever</i>	Integer
<i>median_playtime_forever</i>	Integer
<i>developers</i>	Array
<i>publishers</i>	Array
<i>categories</i>	Array
<i>genres</i>	Array
<i>tags</i>	Array
<i>notes</i>	Text

Table 7. Fields indexed in Solr, their field types, and properties.

Field Name	Indexed Type	MultiValued
<i>title</i>	text_general_game	No
<i>about_the_game</i>	text_general_game	No
<i>supported_languages</i>	text_general_game	Yes
<i>full_audio_languages</i>	text_general_game	Yes
<i>developers</i>	text_general_game	Yes
<i>publishers</i>	text_general_game	Yes
<i>categories</i>	text_general_game	Yes
<i>genres</i>	text_general_game	Yes
<i>tags</i>	text_general_game	Yes
<i>age_rating_label</i>	text_general_game	Yes
<i>release_year_label</i>	text_general_game	No
<i>price_range_label</i>	text_general_game	No
<i>OS_label</i>	text_general_game	Yes
<i>playtime_label</i>	text_general_game	No
<i>reviews</i>	text_general_game	No
<i>price</i>	float_field	No
<i>positive</i>	int_field	No
<i>negative</i>	int_field	No
<i>required_age</i>	int_field	No
<i>release_date</i>	date_field	No
<i>text</i>	text_general_game	Yes
<i>embedding_vector</i>	knn_vector_384	No

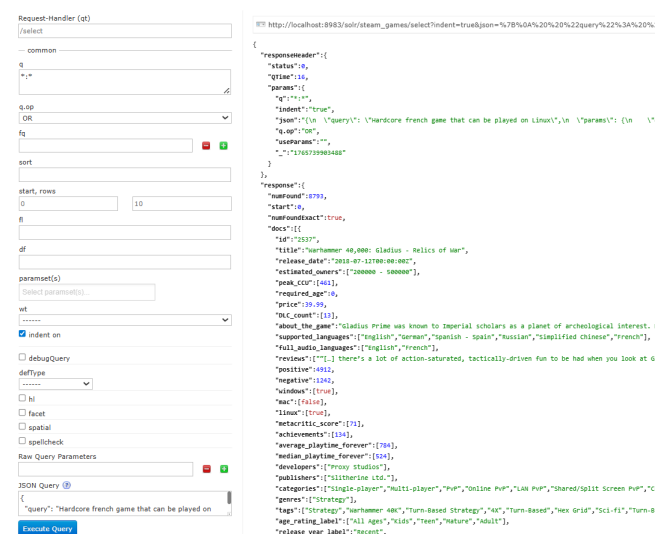


Figure 11. Result of executing query 1

Request-Handler (qt)

/select

common

q

OR

fq

sort

start_rows

0 10

#

df

param(s)

Select param(s)

wt

indent on

debugQuery

defType

hl

facet

spatial

spellcheck

Raw Query Parameters

JSON Query

{

"query": "Cheap adventure games for Windows",

Escudo Query

http://localhost:8983/solr/steam_games/select?indent=true&json=%7B%0A%20%22query%22%3A%20%22

```

{
  "responseHeader": {
    "status": 0,
    "qtime": 189,
    "params": {
      "q": "*",
      "indent": "true",
      "json": "{\n  'query': 'Cheap adventure games for windows',\n  'params': {\n    'defType': 'edismax',\n    'q.op': 'OR',\n    'useParams': ''\n  },\n  '': '1765739903408'\n}",
      "useParams": ""
    }
  },
  "response": {
    "numFound": 20000,
    "start": 0,
    "numFoundExact": true,
    "docs": [
      {
        "id": "14801",
        "title": "DOD Zephyr Lite Steam Edition",
        "release_date": "2016-02-27T00:00:00Z",
        "estimated_owners": [0 - 20000],
        "peak_ccp": [1],
        "required_age": 0,
        "price": 199.99,
        "dlc_count": [0],
        "about_the_game": "DOD Zephyr allows you to automatically and easily create 3D models from photos - The",
        "supported_languages": ["English", "Italian", "Japanese", "simplified chinese", "German", "Korean", "French", "positive": 10,
        "negative": 0,
        "windows": [true],
        "mac": [false],
        "linux": [false],
        "metacritic_score": [0],
        "achievements": [0],
        "average_playtime_forever": [0],
        "median_playtime_forever": [0],
        "developers": ["orfinow snc"],
        "publishers": ["orfinow snc"],
        "categories": ["Steam Trading Cards"],
        "genres": ["Animation & Modeling", "Design & Illustration", "Video Production"],
        "tags": ["Video production", "Animation & Modeling", "Design & Illustration", "Game Development"],
        "age_rating_label": ["All Ages", "Kids", "Teen", "Mature", "Adult"],
        "release_year_label": "Recent",
        "price_range_label": "Expensive",
        "os_label": ["Windows"]
      }
    ]
  }
}

```

Figure 12. Result of executing query 2

Request-Handler (qt)

/select

common

q

OR

fq

sort

start_rows

0 10

#

df

param(s)

Select param(s)

wt

indent on

debugQuery

defType

hl

facet

spatial

spellcheck

Raw Query Parameters

JSON Query

{

"query": "Recent and expensive games from Ubisoft",

Escudo Query

http://localhost:8983/solr/steam_games/select?indent=true&json=%7B%0A%20%22query%22%3A%20%22

```

{
  "responseHeader": {
    "status": 0,
    "qtime": 13,
    "params": {
      "q": "*",
      "indent": "true",
      "json": "{\n  'query': 'Recent and expensive games from Ubisoft',\n  'params': {\n    'defType': 'edismax',\n    'q.op': 'OR',\n    'useParams': ''\n  },\n  '': '1765739903408'\n}",
      "useParams": ""
    }
  },
  "response": {
    "numFound": 18750,
    "start": 0,
    "numFoundExact": true,
    "docs": [
      {
        "id": "13738",
        "title": "Watch Dogs 2",
        "release_date": "2016-11-28T00:00:00Z",
        "estimated_owners": [1000000 - 2000000],
        "peak_ccp": [10],
        "required_age": 15,
        "price": 69.99,
        "dlc_count": [0],
        "about_the_game": "Play as Marcus Holloway, a brilliant young hacker living in the heart of the tech",
        "supported_languages": ["English", "French", "Italian", "German", "Spanish - Spain", "Czech", "Dutch", "Hungarian", "Full audio languages": ["English", "French", "Italian", "German", "Spanish - Spain", "Japanese", "Portuguese - positive": 100,
        "negative": 1752,
        "windows": [true],
        "mac": [false],
        "linux": [false],
        "metacritic_score": [0],
        "achievements": [0],
        "average_playtime_forever": [2000],
        "median_playtime_forever": [848],
        "developers": ["Ubisoft"],
        "publishers": ["Ubisoft"],
        "categories": ["Single-player", "Multi-player", "Co-op", "In-App Purchases", "Partial Controller Support"],
        "genres": ["Action", "Adventure"],
        "tags": ["Open world", "Illuminati", "Hacking", "Action", "Parkour", "Multiplayer", "Stealth", "Third Person", "VR",
        "age_rating_label": ["All Ages", "Mature"],
        "release_year_label": "Recent",
        "price_range_label": "Expensive"
      }
    ]
  }
}

```

Figure 14. Result of executing query 4

```

{
  "query": "Hardcore french game that can be played on Linux",
  "fields": "id,score",
  "params": {
    "defType": "edismax",
    "qf": "OS_label^4 supported_languages^3 playtime_label^2 full_audio_languages",
    "fq": [
      "OS_label:Linux",
      "playtime_label:Hardcore"
    ]
  }
}

```

Figure 15. Boosted JSON query of the first information need

```

{
  "query": "Cheap adventure games for Windows",
  "fields": "id,score",
  "params": {
    "defType": "edismax",
    "qf": "price_range_label^4 OS_label about_the_game^3 tags^3",
    "boost": "sum(termfreq(about_the_game,*adventure), termfreq(tags,adventure*))"
  }
}

```

Figure 16. Boosted JSON query of the second information need

Request-Handler (qt)

/select

common

q

OR

fq

sort

start_rows

0 10

#

df

param(s)

Select param(s)

wt

indent on

debugQuery

defType

hl

facet

spatial

spellcheck

Raw Query Parameters

JSON Query

{

"query": "casual single player zombie games",

Escudo Query

http://localhost:8983/solr/steam_games/select?indent=true&json=%7B%0A%20%22query%22

```

{
  "responseHeader": {
    "status": 0,
    "qtime": 188,
    "params": {
      "q": "*",
      "indent": "true",
      "json": "{\n  'query': 'casual single player zombie games',\n  'params': {\n    'defType': 'edismax',\n    'q.op': 'OR',\n    'useParams': ''\n  },\n  '': '1765739903408'\n}",
      "useParams": ""
    }
  },
  "response": {
    "numFound": 13990,
    "start": 0,
    "numFoundExact": true,
    "docs": [
      {
        "id": "16119",
        "title": "Strategy - Single Player",
        "release_date": "2016-12-18T00:00:00Z",
        "estimated_owners": [0 - 20000],
        "peak_ccp": [1],
        "required_age": 0,
        "price": 6.99,
        "dlc_count": [0],
        "about_the_game": "Play Strategy® Official - Single Player from Jumbo in cooperation with vo",
        "supported_languages": ["English"],
        "positive": 13,
        "negative": 0,
        "windows": [true],
        "mac": [false],
        "linux": [false],
        "metacritic_score": [0],
        "achievements": [0],
        "average_playtime_forever": [0],
        "median_playtime_forever": [0],
        "developers": ["OrangeGames"],
        "publishers": ["OrangeGames", "CloudGames"],
        "categories": ["Single-player", "Steam Achievements"],
        "genres": ["Strategy"],
        "tags": ["Strategy", "Singleplayer", "Board game"],
        "age_rating_label": ["All Ages", "Kids", "Teen", "Mature", "Adult"],
        "release_year_label": "Recent",
        "price_range_label": "Cheap",
        "os_label": ["Windows"]
      }
    ]
  }
}

```

Figure 13. Result of executing query 3

```
{
  "query": "casual single player zombie games",
  "fields": "id,score",
  "params": {
    "defType": "edismax",
    "qf": "about_the_game^3 categories^4 playtime_label^3 title",
    "pf": "\\\"zombie singleplayer\\\"~10^6",
    "boost": "
      sum(termfreq(categories,single*),termfreq(about_the_game,
      zombie))"
  }
}
```

Figure 17. Boosted JSON query of the third information need

```
{
  "query": "recent and expensive games from Ubisoft",
  "fields": "id,score",
  "params": {
    "defType": "edismax",
    "qf": "developers^2 publishers^3 release_year_label^2
      price_range_label^5",
    "boost": "
      sum(termfreq(developers,Ubisoft),termfreq(publishers,Ubisoft))",
    "fq": [
      "release_year_label:Recent",
      "price_range_label:Expensive"
    ]
  }
}
```

Figure 18. Boosted JSON query of the fourth information need

Table 8. Performance comparison between Simple (Q1) and Boosted (Q2) queries.

Information Need	AP		AUC		P@10	
	Q1	Q2	Q1	Q2	Q1	Q2
1. Hardcore / Linux	[0.483]	[0.688]	[0.530]	[0.716]	[0.500]	[0.700]
2. Cheap / Windows	[0.020]	[0.560]	[0.036]	[0.600]	[0.100]	[0.600]
3. Casual / Zombie	[0.100]	[0.480]	[0.136]	[0.556]	[0.200]	[0.700]
4. Recent / Ubisoft	[0.581]	[1.000]	[0.691]	[1.000]	[0.800]	[1.000]

Table 9. Retrieval metrics comparison between Simple (Q1) and Boosted (Q2) queries.

Query	Num Ret		R		NR		RR	
	Q1	Q2	Q1	Q2	Q1	Q2	Q1	Q2
1. Hardcore / Linux	10	10	1	5	9	5	0.2000	1.0000
2. Cheap / Windows	10	10	6	7	4	3	1.0000	1.0000
3. Casual / Zombie	10	10	2	7	8	3	0.5000	0.5000
4. Recent / Ubisoft	10	10	8	10	2	0	1.0000	1.0000

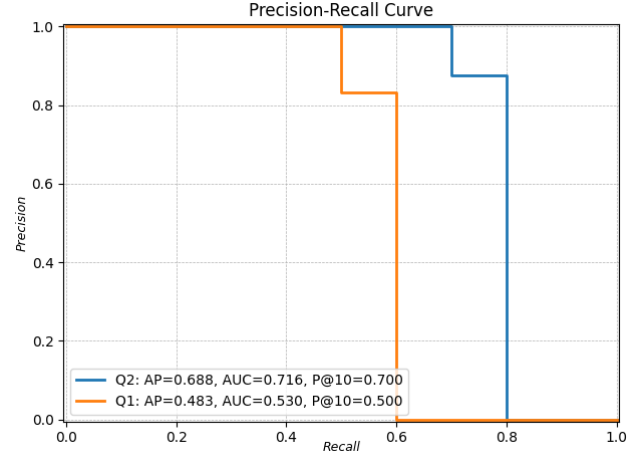


Figure 19. Precision-recall curve of first simple and boosted queries (Q1 is simple and Q2 is boosted) for Hardcore french game that can be played on Linux information need

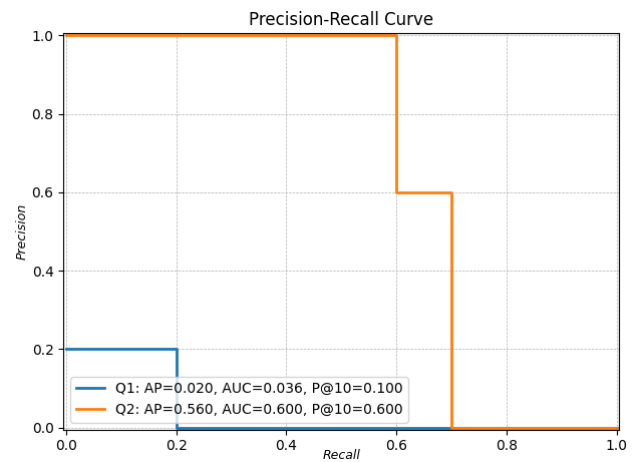


Figure 20. Precision-recall curve of second simple and boosted queries (Q1 is simple and Q2 is boosted) for Cheap adventure games for Windows information need

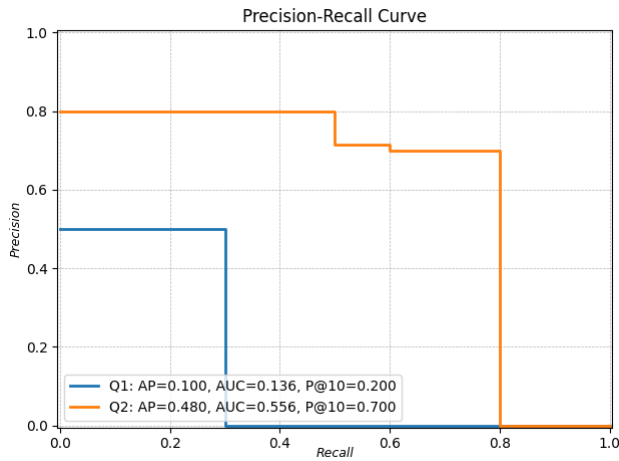


Figure 21. Precision-recall curve of third simple and boosted queries (Q1 is simple and Q2 is boosted) for Casual single player zombie games information need

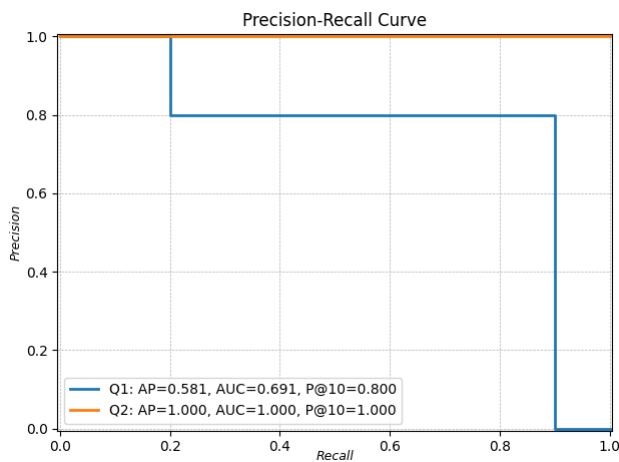


Figure 22. Precision-recall curve of fourth simple and boosted queries (Q1 is simple and Q2 is boosted) for Re-cent and expensive games from Ubisoft information need

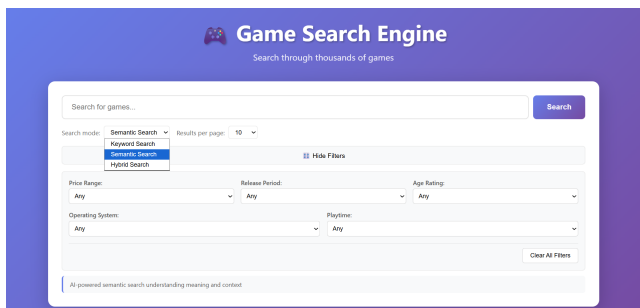


Figure 23. Print of the search section of the GUI interface

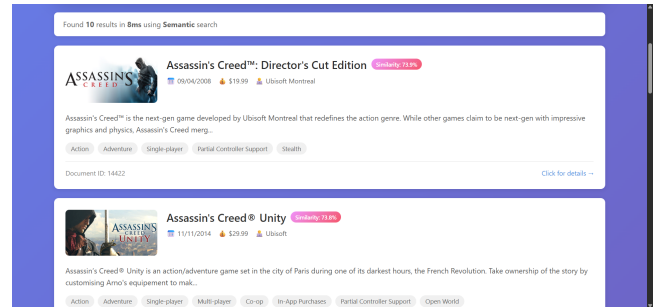


Figure 24. Print of the results section of the GUI interface

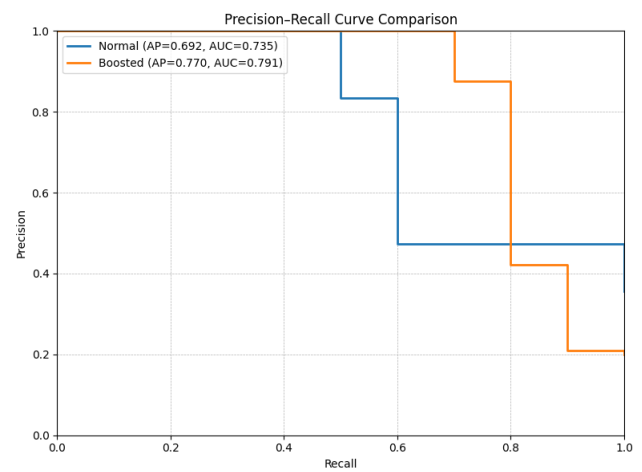


Figure 25. Precision-recall curve of first query with all 4 types of retrieval, query: hardcore french Linux

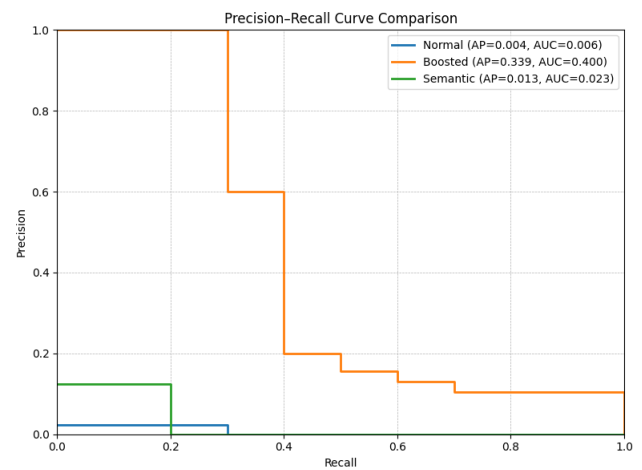


Figure 26. Precision-recall curve of second query with all 4 types of retrieval, query: cheap adventure Windows

Table 11. Retrieval diagnostics for the new system evaluation (top 10 results).

Query	Run	Num Ret	R	NR	RR
1. hardcore french linux	Normal	10	2	8	0.2000
1. hardcore french linux	Boosted	10	4	6	1.0000
1. hardcore french linux	Semantic	10	0	10	0.0000
1. hardcore french linux	Hybrid	10	0	10	0.0000
2. cheap adventure windows	Normal	10	2	8	0.0118
2. cheap adventure windows	Boosted	10	9	1	1.0000
2. cheap adventure windows	Semantic	10	1	9	0.1250
2. cheap adventure windows	Hybrid	10	0	10	0.0000
3. casual single player zombie	Normal	10	5	5	0.5000
3. casual single player zombie	Boosted	10	9	1	0.5000
3. casual single player zombie	Semantic	10	2	8	0.1429
3. casual single player zombie	Hybrid	10	2	8	1.0000
4. recent expensive ubisoft	Normal	10	9	1	1.0000
4. recent expensive ubisoft	Boosted	10	10	0	1.0000
4. recent expensive ubisoft	Semantic	10	0	10	0.0000
4. recent expensive ubisoft	Hybrid	10	7	3	0.5000

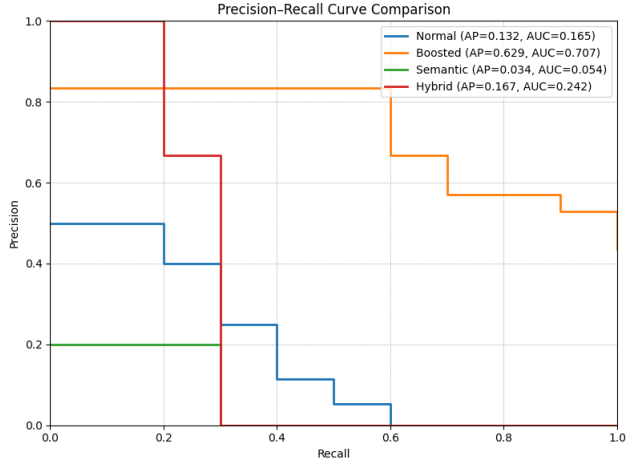


Figure 27. Precision-recall curve of third query with all 4 types of retrieval, query: casual single player zombie

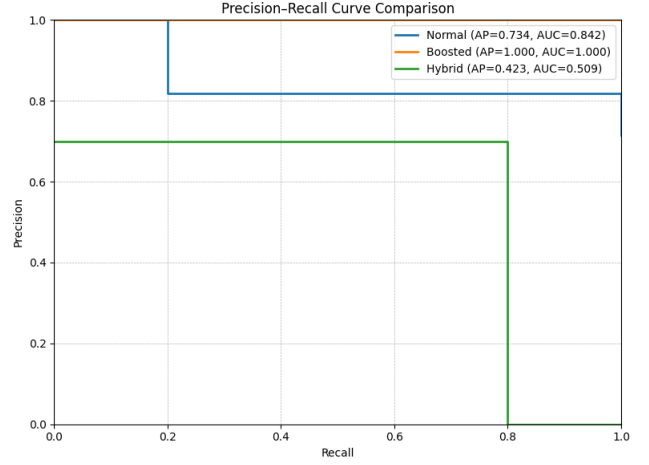


Figure 28. Precision-recall curve of forth query with all 4 types of retrieval, query: recent expensive UbiSoft

Table 10. Performance gap between boosted keyword and semantic search

Metric	Boosted	Semantic	Gap
Average R	8.0	0.75	10.7× worse
Average RR	0.875	0.057	15.4× worse
Queries with R=0	0/4	2/4	2 failures